

Plan de Diseño y Manual Técnico del Sistema de Gestión de Taller Automotriz

El presente documento es un **blueprint** (plano de arquitectura) y manual técnico para construir un sistema integral de gestión de un taller automotriz. El objetivo es detallar todos los módulos y componentes necesarios, su interacción, la tecnología a utilizar y las consideraciones para implementar la solución al 100%. Se busca una solución completa, probada y lista para desplegar en entorno Ubuntu (sin uso de Docker, instalado directamente en el sistema operativo), con un instalador empaquetado en ZIP. A continuación, se describen la arquitectura propuesta, cada módulo funcional, y las pautas técnicas para que OpenAI Codex (u otra herramienta de generación de código) pueda construir el proyecto y dejarlo listo en un repositorio GitHub, incluyendo pruebas de funcionamiento.

Tecnologías y Arquitectura del Sistema

Para lograr un sistema robusto, escalable y fácil de mantener, se propone la siguiente arquitectura y stack tecnológico:

- **Arquitectura General:** Aplicación **web** de múltiples módulos integrada en una sola plataforma. Se adoptará una arquitectura de **cliente-servidor**. El servidor proporcionará una API (RESTful o GraphQL) para las operaciones de negocio, y el cliente será una aplicación web interactiva que consume esa API.
- **Backend (Servidor):** Usar un entorno **Ubuntu Linux** con un servidor backend implementado en Node.js (Express) o Python (Django/Flask) según preferencia. Dado que se desea evitar Docker, el backend se instalará directamente en el sistema operativo, manejando procesos y servicios nativos. Se recomienda estructurar el backend de forma modular, separando controladores (endpoints), servicios (lógica de negocio) y modelos de datos.
- **Base de Datos:** Utilizar **PostgreSQL** como sistema gestor de base de datos relacional, por su robustez y fiabilidad. Todas las entidades del sistema (clientes, vehículos, repuestos, facturas, etc.) se almacenarán en PostgreSQL. Se empleará un ORM (por ejemplo, Sequelize si se usa Node, o Django ORM/SQLAlchemy en Python) para facilitar el manejo de datos y mantener la portabilidad. También se establecerán mecanismos de **auditoría** en la base de datos (por ejemplo, triggers o tablas de historial) para registrar quién hizo cada cambio en inventario y otras operaciones críticas.
- **Frontend (Cliente Web):** Desarrollar una aplicación web **moderna** usando HTML/CSS/JavaScript. Se puede emplear un framework como **React**, **Angular** o **Vue** para crear una interfaz de una sola página (SPA) dinámica. Esto permitirá una experiencia interactiva tanto para el portal de clientes ("Garage") como para los módulos internos. El frontend se conectará al backend via API. Se integrará **Babylon.js** (o una biblioteca 3D similar con buen rendimiento y soporte) para visualizar modelos 3D de vehículos en el módulo "Garage".
- **Renderizado 3D:** Para la función de mostrar un modelo 3D del vehículo según VIN, se usará Babylon.js en la web. Babylon.js provee un motor WebGL eficiente con amplia documentación y bibliotecas de soporte para carga de modelos. En caso de encontrar una alternativa más ligera o con

mayores bibliotecas de modelos (por ejemplo **Three.js**), se evaluará su uso, pero Babylon.js es una opción confiable para iniciarlo.

- **Servidor Web & API:** El servidor expondrá endpoints seguros para cada módulo (ej: `/api/inventario`, `/api/facturacion`, `/api/vehiculos`, etc.). Se manejarán autenticación y autorización robustas (por ejemplo, usando JWT para clientes web, con tokens de acceso). Se implementarán control de errores y validaciones en cada endpoint.
- **Multisucursal:** La arquitectura soportará **múltiples sucursales** dentro de la misma base de datos o mediante bases de datos separadas por sucursal. La recomendación es un solo esquema de DB con identificación de sucursal en las tablas relevantes, para facilitar reportes consolidados, pero filtrando por sucursal según usuario. Esto permite aislar los datos por tienda/taller pero mantener la aplicación unificada.
- **Seguridad:** Implementar autenticación de usuarios con **roles** (admin, técnico, cliente, etc.) y autorización a nivel de funcionalidades (p. ej., solo usuarios con rol de técnico pueden acceder al módulo técnico). Las contraseñas se almacenarán cifradas (hash + salt). Además, se considerarán medidas de seguridad como validación de inputs en servidor y cliente para prevenir inyección SQL, XSS, CSRF, etc.
- **Cumplimiento Legal (Honduras - SAR):** El sistema de facturación estará adaptado a la normativa hondureña (Servicio de Administración de Rentas, SAR). Esto implica manejar rangos de folios/ autorizaciones de facturas preimpresas o CAI, formatos fiscales requeridos y cálculos de impuestos locales (como ISV, equivalente al IVA). La arquitectura debe permitir ajustar la generación de facturas al formato oficial.
- **Despliegue:** Se proporcionará un **script de instalación** (por ejemplo, `install.sh`) que configure en Ubuntu todos los componentes: instalación de PostgreSQL (si no está presente), creación de la base de datos y usuarios, instalación de dependencias del backend (via npm/pip), compilación del frontend, carga de datos iniciales (por ejemplo, datos de ejemplo o catálogos básicos), y arranque del servidor. Este script facilitará desplegar el sistema en nuevas máquinas sin Docker. Para multiplataforma, se puede documentar también los pasos para Windows (instalación de Node, Python, Postgres), aunque el entorno principal objetivo es Ubuntu.
- **Repositorio y CI/CD:** El código fuente se almacenará en GitHub, estructurado en un monorepo o repos separados (frontend/backend). Se incluirán instrucciones de uso en el README. Sería ideal configurar integración continua para correr pruebas automáticas cuando se hagan cambios. Dado que se busca un sistema probado, se agregarán pruebas unitarias y de integración en el repositorio. Codex debería generar estos test junto con el código principal para garantizar que todo funcione como se espera.

Módulo de Inventario (Gestión de Repuestos y Partes)

El **módulo de inventario** es crucial, ya que manejará los repuestos, piezas y suministros del taller. Este módulo permitirá registrar y hacer seguimiento de cada parte disponible para venta o uso en reparaciones, incluyendo su ubicación en bodegas o sucursales, cantidad en existencia y detalles relevantes. Las funcionalidades principales del módulo de inventario son:

- **Registro de Productos:** Alta de nuevos repuestos/partes en el sistema con campos como código o SKU, nombre/descripción, marca, modelo/compatibilidad (por ejemplo, a qué vehículos aplica), proveedor, costo, precio de venta, ubicación (bodega/estante) y stock inicial. Si ya existe un catálogo base, los repuestos pueden cargarse masivamente o seleccionarse de una lista.

- **Entrada y Salida de Inventario:** Registrar entradas de stock (por compras o retornos) y salidas de stock (ventas directas o uso en servicio). Cada movimiento debe quedar registrado con fecha, usuario responsable y concepto (ej. "Compra a Proveedor X", "Salida por reparación orden #123"). Esto permite auditoría detallada de inventario.
- **Actualización y Ajustes:** Posibilidad de ajustar manualmente el inventario (con las debidas autorizaciones) en caso de auditorías físicas, daños o pérdidas. Estos ajustes deben quedar guardados con justificación.
- **Consultas y Búsqueda:** Permitir buscar repuestos por código, nombre o incluso filtrar por vehículo compatible. Por ejemplo, el técnico o vendedor puede buscar piezas que sean compatibles con un determinado modelo de auto.
- **Alerta de Stock Mínimo:** Definir un nivel mínimo de inventario para cada producto y alertar cuando la cantidad disponible cae por debajo de ese umbral. El sistema podría mostrar notificaciones o reportes de "stock bajo", facilitando al encargado hacer pedidos a tiempo.
- **Gestión de Proveedores:** (Opcional dentro de inventario o en módulo separado) Mantener un listado de proveedores asociados a cada repuesto, con datos de contacto, para agilizar reordenar stock. Podría incluir histórico de compras por proveedor.
- **Múltiples Almacenes/Sucursales:** Dado que habrá un módulo de sucursales, el inventario debe manejarse por ubicación. Cada pieza tendrá cantidades separadas por sucursal/almacén. Las transferencias entre sucursales deben ser soportadas: ej. mover X repuestos de la sucursal A a la B, registrando salida de A y entrada a B.
- **Integración con Otros Módulos:** El inventario se vincula con **facturación** (al vender una parte se reduce stock), con **módulo técnico** (al usar piezas en una reparación, deben descontarse o reservarse del inventario) y con **contabilidad** (control del costo de las mercancías vendidas, valoración de inventario). Estos enlaces se detallarán en la integración, pero a nivel de diseño, cada transacción relevante en otros módulos debería reflejarse aquí.
- **Auditoría:** Cada modificación significativa (entrada, salida, ajuste) llevará registro de qué usuario la realizó y cuándo, para fines de auditoría interna. Esto se puede lograr con campos de tracking en la base de datos (usuario_creacion, fecha_creacion, etc.) y tablas de historial.

Módulo de Mano de Obra (Gestión de Trabajo y Tarifas)

El módulo de **Mano de Obra** se enfoca en gestionar el trabajo de taller en términos de tareas realizadas por los técnicos, tiempo invertido y costo de la labor. Es complementario al módulo técnico, pero aquí nos centramos en cómo definir y cobrar la mano de obra independiente de las partes. Principales características:

- **Catálogo de Servicios/Mano de Obra:** Definir una lista de tareas o servicios típicos que ofrece el taller con un precio base o tarifa por hora. Ejemplos: cambio de aceite, alineación, reemplazo de frenos, etc., cada uno con un costo estándar o tiempo estimado. Esto sirve para que al generar una orden de trabajo se puedan agregar ítems de mano de obra fácilmente.
- **Registro de Horas Trabajadas:** Para trabajos específicos no estandarizados, el técnico puede registrar el tiempo empleado (por ejemplo, 3 horas de diagnóstico eléctrico). El sistema debería permitir ingresar horas o unidades de labor y calcular el costo en base a la tarifa por hora definida para ese tipo de trabajo o técnico.
- **Asignación de Técnicos:** Asociar técnicos responsables a cada tarea o servicio realizado. Se puede registrar quién realizó qué trabajo, para control interno y posiblemente para mostrar al cliente quién atendió su vehículo.

- **Cálculo de Costos de Mano de Obra:** Integrado con facturación, de modo que las tareas de mano de obra aprobadas se sumen en la orden de servicio y luego en la factura. Si el técnico añade trabajos adicionales durante la reparación, estos pueden actualizar el costo total a cobrar (previa aprobación del cliente si es algo no presupuestado).
- **Historial de Mano de Obra:** Mantener un historial por vehículo de todas las labores realizadas. Esto complementa el historial del vehículo: no solo qué se cambió (piezas), sino también qué trabajos se le han hecho y cuándo, con qué kilometraje, etc. Útil para futuras referencias y para que el cliente lo vea en el portal.
- **Integración con Módulo Técnico:** El módulo técnico (descrito más adelante) genera órdenes de trabajo que incluirán mano de obra requerida. Este módulo de mano de obra proporciona los elementos (servicios predefinidos, tarifas, etc.) para que el técnico los seleccione y agregue al presupuesto. Al aprobarse y completarse el trabajo, se descuenta tiempo disponible si se lleva control de horarios, y se registra el costo final en la factura.
- **Evaluación de Desempeño (Opcional):** Con los datos de mano de obra, se podría sacar métricas por técnico (tiempo promedio por tarea, tareas completadas por mes, etc.), aunque esto es más para reportes internos. Si se desea, se incluiría en el módulo de gráficos o reportes una sección de eficiencia de mano de obra.

Módulo "Garage" (Portal del Cliente con Vehículo y Repuestos)

El módulo **Garage** es una interfaz orientada al cliente, donde cada cliente puede iniciar sesión, registrar su vehículo y realizar diversas acciones como ver historiales, aprobar trabajos y comprar repuestos específicos para su coche. Este portal mejora la experiencia del cliente al darle visibilidad y capacidad de interacción con el taller. Sus componentes más importantes son:

- **Registro e Inicio de Sesión del Cliente:** Los clientes podrán crear una cuenta en la plataforma, o ser dados de alta por un administrador que les proporciona acceso. Durante el registro, se pedirá información básica y, crucialmente, el **VIN (Vehicle Identification Number)** de su vehículo (o vehículos) para vincularlos a su perfil. La autenticación debe ser segura y sencilla (correo/contraseña, con opción de OAuth Google/Facebook si se desea en el futuro, pero inicialmente usuario y contraseña está bien).
- **Perfil del Vehículo:** Una vez registrado el VIN, el sistema buscará o creará una entrada del vehículo en la base de datos central de vehículos. Idealmente, el sistema podría decodificar el VIN para obtener marca, modelo, año, motor, etc., utilizando alguna librería o API de decodificación VIN. Con esos datos, se seleccionará un modelo 3D representativo del auto.
- **Visualización 3D del Vehículo:** En la sección "Garage" del portal, el cliente verá un modelo 3D de su coche en un pedestal (como en un showroom virtual). Para implementar esto:
 - Se integrará **Babylon.js** en la página para renderizar un modelo 3D. Inicialmente, se puede utilizar un modelo genérico o representativo de la marca/modelo. Por ejemplo, si el cliente tiene un Toyota Corolla 2018, cargar un modelo 3D de un Corolla similar. Si no se dispone del modelo exacto, usar uno genérico que luego podría refinarse.
 - Cada pieza o sección importante del modelo 3D debería ser **interactiva**. Es decir, al hacer clic en, digamos, la llanta/freno, podría resaltar y mostrar los repuestos asociados (discos, pastillas de freno disponibles en inventario para ese modelo).
- **Asociación de piezas con el modelo 3D:** Se requiere un mapeo entre partes del modelo 3D y los repuestos en el inventario. Esto implica tener identificadores de piezas del vehículo que correspondan con los SKU en inventario. Inicialmente, se puede empezar con las piezas más

comunes (filtros, aceite, frenos, baterías) para demostrar la idea. Este mapeo puede ampliarse con el tiempo.

- El cliente podría navegar por su vehículo virtual, seleccionar una parte, y el sistema le mostrará opciones de repuesto para esa parte disponibles en la tienda (con precios).
- **Compra de Repuestos en Línea:** Desde el mismo portal, el cliente debería poder añadir repuestos a un carrito de compra y proceder a comprarlos si así lo desea (similar a un pequeño e-commerce enfocado en piezas para su auto). Integrar un flujo de pedido:
- Carrito de compras de repuestos específicos (limitado a aquellos compatibles con su vehículo, para evitar confusiones).
- Generación de orden de venta, pago en línea (si se planea implementar pagos en línea vía tarjeta, PayPal u otros, o simplemente reservación para pago en tienda).
- Reducción de inventario al confirmarse la venta, y posibilidad de recoger en tienda o envío (logística de envío puede ser algo a definir, pero se puede dejar preparado).
- **Historial del Vehículo:** El cliente podrá acceder a un historial donde se listan:
 - Servicios realizados anteriormente en el taller para ese vehículo (con fechas, qué se hizo, kilometraje, costos).
 - Repuestos comprados antes (con fecha y factura).
 - Recomendable incluir documentos adjuntos por servicio, p.ej. diagnóstico o resultados de revisión, fotos, etc., si el técnico los subió.
- **Aprobación de Presupuestos:** Integrado con el módulo técnico: cuando el técnico genera un diagnóstico y lista de reparaciones con sus costos (presupuesto o cotización), el cliente debería recibir una notificación (por email y/o al ingresar al portal) y ver el detalle en su cuenta. Desde allí puede **aprobar o rechazar** cada trabajo o el conjunto:
 - Si aprueba, esa aprobación queda registrada y el taller procede con la reparación.
 - Si rechaza alguna parte del presupuesto, el taller podría ajustar o no realizar ese trabajo.
 - Esta interacción ahorra tiempo en llamadas y asegura un registro formal de la autorización del cliente.
- **Recordatorios y Promociones (Marketing):** El portal también mostrará notificaciones o mensajes promocionales personalizados (integración con el módulo de marketing). Por ejemplo, un aviso de "Su vehículo está próximo a necesitar cambio de aceite, le ofrecemos 10% de descuento si agenda en las próximas 2 semanas". Estos mensajes pueden mostrarse en un panel o sección de notificaciones dentro del perfil del cliente.
- **Experiencia de Usuario:** El diseño del portal debe ser claro y amigable para usuarios no técnicos. Debe funcionar tanto en escritorio como en dispositivos móviles (diseño responsive), ya que muchos clientes querrán acceder desde su smartphone para ver el estado de su vehículo o aprobar trabajos.
- **Seguridad del Portal:** Asegurar que cada cliente solo acceda a la información de sus propios vehículos y datos. Esto significa que las APIs que entregan datos del vehículo o historial deben verificar el token de usuario y filtrar por su ID de cliente. También, proteger datos sensibles (como VIN, dirección del cliente) cumpliendo normas básicas de privacidad.

Nota: Para la primera versión, se puede implementar la visualización 3D con un modelo genérico y una interacción limitada (por ejemplo, algunas partes clicables de demostración). Es entendible que la funcionalidad completa de mapa de todas las piezas es compleja; por lo tanto, se sugiere **incluir un modelo 3D básico quizás obtenido de un proyecto previo (por ejemplo, un modelo usado en un taller de San Pedro Sula) como demostración**. Este modelo parcialmente completo servirá de base y en el futuro se podrán añadir más detalles o modelos específicos por VIN. Lo importante es sentar la estructura para soportar esta característica innovadora.

Módulo de Facturación (Sistema de Facturas y Cobros)

El módulo de **Facturación** gestionará la emisión de facturas por ventas de repuestos y por servicios del taller, asegurándose de cumplir con la normativa fiscal de Honduras (SAR) y reflejando el formato de factura preimpresa tradicional. Este módulo debe permitir crear, personalizar, imprimir y registrar facturas de manera profesional. Sus aspectos clave son:

- **Emisión de Facturas de Venta:** Cada vez que se realiza una venta (sea venta de repuestos en mostrador/portal o cobro de servicios de reparación), el sistema generará una factura electrónica con número único. Las facturas incluirán datos fiscales de la empresa (Nombre, RTN, dirección, etc.), datos del cliente (nombre, identidad o RTN si es empresa, dirección), fecha y detalle de los productos/servicios vendidos con cantidades, precios unitarios, subtotales, impuestos y total.
- **Formato Preimpreso Personalizable:** Se deberá replicar el formato de la factura preimpresa oficial. Esto implica:
 - Ubicar los elementos (logo, datos de empresa, columnas de detalle, totales) en un diseño similar a los talonarios autorizados.
 - Incluir campos obligatorios como el número de CAI (Código de Autorización de Impresión) si aplica, el rango autorizado de la factura (ej: del #000-001-01-00000001 al #000-001-01-00000500), fecha límite de emisión, etc., conforme a requisitos del SAR.
 - Posibilidad de ajustar el diseño en caso de cambios regulatorios o de formato (por eso conviene generar la factura con una plantilla que se pueda modificar sin tocar lógica, por ejemplo usando HTML/CSS a PDF o un motor de plantillas para PDF).
- **Cálculo de Impuestos:** Calcular automáticamente el **ISV (Impuesto sobre Ventas)** u otros impuestos aplicables sobre los productos/servicios según la ley hondureña (por ejemplo 15% estándar, u otro porcentaje si hay exenciones o categorías diferentes). Desglosar los impuestos en la factura como corresponde. También contemplar campos de descuentos o exoneraciones si fueran necesarias.
- **Facturación de Servicios vs Productos:** El sistema debe manejar ambos. Cuando se factura una reparación de vehículo, habrá partidas de mano de obra y partidas de repuestos:
 - Las **partidas de mano de obra** pueden estar exentas de ciertos impuestos dependiendo de la legislación, o tener su propio tratamiento. Por lo general, el servicio también lleva ISV, pero hay que confirmarlo según SAR.
 - Las **partidas de repuestos** son bienes con ISV. En la factura se verán ambos tipos de concepto. Debe mostrarse claramente la descripción (ej. "Servicio de cambio de frenos – Mano de Obra" y "Disco de freno marca X – Repuesto").
- **Numeración y Control Fiscal:** Permitir configurar series de facturación. En Honduras, normalmente las empresas tienen rangos autorizados por sucursal o por punto de emisión. El sistema debe:
 - Almacenar el próximo número de factura a emitir, según la serie vigente.
 - Bloquear emisión si se alcanza el fin de la serie, alertando que se debe cargar una nueva autorización.
 - Impedir saltos o duplicados en la numeración (vital para control fiscal).
 - Generar reportes de facturación mensual para el SAR (por ejemplo, libro de ventas).
- **Impresión y PDF:** Cada factura deberá poder imprimirse en tamaño estándar (por ejemplo, tamaño carta o formulario continuo según se use). Además, generar un **PDF** de la factura que se pueda descargar o enviar por email al cliente. El PDF seguirá el formato preimpreso mencionado. Es útil que el sistema permita incluir un código QR o código de barras en la factura con la información del

folio, para verificaciones rápidas (algunas legislaciones lo sugieren; SAR recientemente ha implementado factura electrónica en línea, podría considerarse).

- **Anulaciones y Notas de Crédito:** Incorporar proceso para anular facturas (siempre registrando motivo y referenciando la factura original, generando nota de crédito si ya fue reportada). Esto también debe respetar la normativa (por ejemplo, si no se pueden borrar facturas, solo anular con documento de respaldo).
- **Integración con Contabilidad:** Al generar una factura, además de afectar inventario, debería crear los asientos contables correspondientes (venta contra inventario, impuestos por pagar, cuentas por cobrar si es crédito, etc., detallado en módulo contabilidad). Si ese nivel de detalle es muy avanzado, al menos dejar preparado el enlace o exportación de datos para contabilidad.
- **Multi-sucursal en Facturación:** Si cada sucursal tiene su propia serie de facturación autorizada, el sistema debe manejarlo. Al emitir factura seleccionar la sucursal/punto de venta adecuado (quizás según el usuario que factura o ubicación actual) y usar la numeración correspondiente.
- **Reportes de Ventas:** Permitir listar facturas por rango de fecha, sucursal, cliente, etc., mostrando totales y permitiendo exportar a Excel/CSV. Este reporte servirá para análisis interno y obligaciones fiscales.
- **Seguridad y Permisos:** Solo personal autorizado (por ejemplo cajeros, gerentes) deben poder generar facturas. Usuarios clientes en el portal podrían generar una "pre-factura" o pedido, pero la factura fiscal oficial la emite el sistema con control interno. Implementar roles de modo que un técnico no emita factura, sino que pase la información a facturación.

Módulo de Contabilidad (Compras, Ventas y Finanzas)

El módulo de **Contabilidad** abarca el registro de movimientos financieros del negocio, incluyendo las compras de insumos para reventa, las ventas realizadas (desde facturación) y, opcionalmente, otros aspectos contables básicos. Aunque no pretende ser tan completo como un software contable profesional, debe dar visibilidad de costos, ingresos y utilidades, y soportar las operaciones de compra/venta. Funcionalidades principales:

- **Gestión de Compras (Inventario):** Registrar las compras de productos destinados a inventario. Esto implica crear **órdenes de compra** o ingresar **facturas de proveedores**:
- Registrar proveedor, fecha, número de factura de proveedor, y lista de ítems comprados (ligados a productos del inventario o nuevos productos a crear).
- Actualizar el stock de inventario según lo recibido. Idealmente, la actualización de stock podría estar pendiente hasta que se marque la compra como recibida.
- Registrar el costo de compra por unidad para luego calcular margen de venta.
- Permitir agregar gastos adicionales en la compra (p. ej. flete, aduana) para tener el costo real de la mercancía.
- **Cuentas por Pagar:** Opcionalmente, manejar las cuentas por pagar a proveedores. Si no se paga una compra al contado, registrar la deuda y luego permitir registrar pagos parciales o totales. Esto es un plus contable que se puede considerar según alcance.
- **Ventas e Ingresos:** Integración directa con el módulo de facturación:
- Cada factura emitida se registra en contabilidad como una venta. Si la venta fue al crédito, registrar cuenta por cobrar; si fue de contado, registrar entrada de efectivo.
- Llevar un libro de ventas acumulado, sumando subtotales, impuestos y totales, para ver los ingresos en un período.

- **Cuentas por Cobrar:** Si se permite ventas a crédito (clientes que pagan después, como convenios empresariales), entonces manejar los **créditos**:
 - Registrar el cliente deudor, la factura, el monto y fecha de vencimiento.
 - Permitir registrar pagos de ese cliente, reduciendo la cuenta por cobrar.
 - Alertar de facturas vencidas.
- **Registro de Gastos/Operativos:** Más allá de compras de inventario, tal vez se quiera registrar otros gastos del taller (ej. salarios, servicios públicos, renta). Se podría incluir un submódulo sencillo para gastos:
 - Ingresar fecha, categoría de gasto, proveedor/beneficiario, monto, descripción.
 - Esto ayuda a calcular utilidades netas en reportes.
- **Libro Diario / Asientos:** Dependiendo del alcance contable, se podría generar automáticamente asientos contables básicos:
 - Venta: Cargo a Caja/Bancos, abono a Ingresos y abono a Impuestos por pagar.
 - Costo de venta: Cargo a Costo de Ventas, abono a Inventario (por la salida de inventario, usando método FIFO o promedio).
 - Compra inventario: Cargo a Inventario, abono a Cuentas por pagar (o a Bancos si se pagó al contado).
 - Pago de gasto: Cargo a Gasto X, abono a Caja/Bancos. Esto ya es un nivel más detallado; si Codex lo permite, se puede implementar. Si no, al menos llevar track de totales.
- **Estados Financieros Básicos:** Con los datos de ingresos y gastos, proporcionar **reportes financieros** simplificados:
 - **Reporte de Utilidad:** total ventas - costo de ventas = utilidad bruta; utilidad bruta - gastos = utilidad neta en un rango de tiempo.
 - **Reporte de inventario valorizado:** cantidad en stock * costo promedio, por cada producto, y total.
 - **Reporte de cuentas por pagar/cobrar:** listado de deudas pendientes.
- **Integración con Otros Módulos:** Contabilidad actúa como receptor de datos:
 - Del módulo de facturación recibe ventas (y pagos).
 - Del inventario (compras actualizan inventario y costo).
 - Del módulo de mano de obra/técnico, podría obtener la contabilización del costo de la mano de obra como gasto (por ejemplo, si se pagan horas extra, etc., aunque eso puede manejarse externamente).
 - Con el módulo de gráficos para mostrar indicadores financieros (ganancias, etc.).
- **Seguridad:** Solo perfiles de **Administrador o Contador** deberían acceder completo a este módulo, dado que contiene información sensible financiera. Se pueden dar accesos parciales (por ejemplo, alguien puede ingresar compras pero no ver ciertas estadísticas de gastos).
- **Cumplimiento Local:** Considerar si hay que sacar informes específicos para la autoridad fiscal (por ejemplo, el formato de libro diario, o integraciones con sistemas fiscales). Dado que Honduras está avanzando hacia facturación electrónica, tener en mente la posibilidad de emitir formatos electrónicos en el futuro (pero inicialmente es suficiente con documentos locales).

Módulo de Técnico (Gestión de Órdenes de Servicio y Diagnósticos)

El módulo de **Técnico** está dedicado a los mecánicos/técnicos del taller para gestionar la recepción de vehículos, diagnósticos, solicitudes de repuesto y registro de trabajos realizados. Este módulo digitaliza la

tradicional hoja de trabajo y permite comunicación fluida con el cliente y otros departamentos. Sus funciones incluyen:

- **Recepción de Vehículo y Registro de Orden:** Cuando un cliente trae un vehículo al taller sin cita previa (o con cita), el técnico o recepcionista crea una nueva **Orden de Servicio**. Esta orden contendrá:
 - Datos del vehículo (VIN, modelo, placa), que pueden seleccionarse de los vehículos registrados del cliente o ingresar si es nuevo.
 - Kilometraje actual, combustible aproximado, y cualquier observación inicial (estado del vehículo, accesorios, etc.).
 - Motivo de ingreso o problemas reportados por el cliente.
 - Asignación de técnico responsable o equipo.
 - Fecha/hora de ingreso.
- **Diagnóstico y Presupuesto:** El técnico, tras revisar el vehículo, usa el sistema para anotar los **problemas encontrados** y las **recomendaciones de reparación**. Por cada problema/recomendación:
 - Agregar las piezas/repuestos necesarios (seleccionando del inventario, con precios actuales).
 - Agregar la mano de obra requerida (seleccionando del catálogo de servicios o especificando horas estimadas).
 - El sistema calcula un **presupuesto** total desglosado (piezas + mano de obra) para esa orden.
 - El técnico puede adjuntar notas o fotos del daño (útil para justificar al cliente).
 - *Estado de la Orden:* En esta fase, la orden está en estado "Presupuesto Pendiente de Aprobación".
- **Envío de Presupuesto al Cliente:** Integrado con el portal "Garage" y notificaciones:
 - El cliente recibe una alerta (y/o email) de que tiene un nuevo presupuesto para revisar. El técnico podría también imprimir el presupuesto o enviarlo por PDF al correo desde el sistema, si el cliente prefiere.
 - En el sistema, el presupuesto queda marcado como enviado en fecha/hora.
- **Aprobación/Rechazo de Presupuesto:** Una vez el cliente responde (ya sea desde el portal marcando aprobar/rechazar o comunicándose con el taller), el técnico ve la actualización:
 - Si **aprobado**, la orden de servicio cambia a estado "Aprobado/En Proceso" y el técnico puede proceder con la reparación.
 - Si **rechazado** total o parcialmente, el técnico puede modificar la orden (quitar los items no aprobados o ajustar) y posiblemente volver a enviar, o cerrar la orden si el cliente decidió no hacer nada.
 - Todas estas decisiones quedan registradas (quién aprobó, cuándo).
- **Solicitud de Repuestos al Almacén:** Cuando la orden está aprobada y en proceso, el técnico necesita los repuestos. El módulo debería permitir que el técnico genere una **solicitud interna de repuestos** al inventario:
 - Básicamente una lista de las piezas aprobadas que se necesitan.
 - Un encargado de almacén/inventario podría tener una vista de "Piezas solicitadas por técnicos" para prepararlas.
 - Al entregar las piezas al técnico, el encargado confirma la salida, lo que descuenta el inventario. La orden de servicio registra que las piezas fueron provistas.
 - Si alguna pieza no está en stock localmente, se podría marcar como pendiente o solicitar traslado de otra sucursal (integración con sucursales).
- **Ejecución y Cierre de Orden:** El técnico realiza los trabajos. Puede ir marcando tareas como completadas en el sistema, e incluso añadir notas adicionales descubiertas durante la reparación.

- Si surgen trabajos o piezas adicionales no previstas, el técnico puede agregarlas al sistema **como adicionales**. En este caso, podría requerir una nueva aprobación del cliente (según política del taller). El sistema podría notificar al cliente de "Trabajo adicional recomendado durante reparación" para aprobación rápida.
- Una vez terminado todo, el técnico cambia el estado de la orden a "Completado/Pendiente de Facturación".
- **Historial y Entrega:** Al completar, se genera un **informe de servicio** que resume lo realizado, las piezas usadas, mano de obra, etc. Este informe se puede imprimir para entregar al cliente junto con su vehículo. También se almacena en el historial del vehículo (para que el cliente lo vea en el portal).
- El informe puede incluir garantías de piezas/servicio, recomendaciones de cuidado, próxima mantención sugerida, etc.
- **Integración con Facturación:** Cuando el vehículo está listo para entregarse, usualmente se procede a facturar. El módulo técnico debe tener un botón/acción "Enviar a Facturación" que compile todo el costo final (considerando lo aprobado y realizado) y cree la factura en el módulo de facturación:
- Posiblemente abrirá una pantalla de factura ya llena con los datos, para que el cajero/administrativo la revise y la emita oficialmente.
- Después de facturada, la orden de servicio se marca como **Facturada** y cerrada.
- **Seguimiento de Estado:** Tanto para clientes (en portal) como para administración, la orden de servicio debería reflejar estados claros: Recibido -> En diagnóstico -> Enviando presupuesto -> En espera de aprobación -> En reparación -> Listo para facturar -> Finalizado. Esto permite ver en qué paso está cada vehículo en el taller en cualquier momento. Un tablero para ver todas las órdenes activas y su estado sería útil para gestión diaria.
- **Métricas de Taller:** Usando los datos de este módulo, se pueden sacar algunas métricas:
 - Tiempo promedio desde ingreso hasta entrega.
 - Porcentaje de presupuestos aprobados/rechazados.
 - Carga de trabajo por técnico (cuántas órdenes atendió, etc.). Estas métricas pueden enviarse al módulo de gráficos para visualización.
- **Seguridad:** Los técnicos (rol de usuario) solo deben ver las órdenes relacionadas a su trabajo o las abiertas. Un jefe de taller o administrador puede ver todas. El sistema debe evitar que técnicos alteren precios o cosas críticas sin autorización; por ejemplo, quizá solo pueden elegir piezas y servicios, pero no cambiar sus precios (eso viene del inventario y catálogo de servicios). Ciertas acciones, como marcar completado, podrían requerir permisos especiales (o doble verificación) para evitar que se cierre una orden por error.

Módulo de Gráficos y Reportes (Visualización de Indicadores)

El módulo de **Gráficos** proporcionará una visión visual de los datos clave del taller, permitiendo evaluar el desempeño del negocio a través de estadísticas y tendencias. Incluye reportes tanto tabulares como gráficos (barras, líneas, pie charts) sobre diferentes áreas: ventas, ganancias, uso de inventario, eficiencia de técnicos, etc. Funcionalidades previstas:

- **Panel de Control (Dashboard):** Al iniciar sesión un usuario administrativo, se mostrará un resumen gráfico con los indicadores principales:
- **Ventas del Mes:** Gráfico de barras o línea mostrando ventas diarias o acumuladas en el mes actual, comparado con meses anteriores.
- **Ingresos vs Egresos:** Un gráfico que compare ingresos (ventas) contra egresos (compras, gastos) en un periodo, mostrando la utilidad neta.

- **Servicios Realizados:** Cantidad de órdenes de servicio completadas en el periodo, quizás categorizadas por tipo (mantenimiento, reparación mayor, etc.).
- **Piezas Más Vendidas:** Top 5 o Top 10 repuestos más vendidos (en unidades o valor) en el último periodo, posiblemente con un gráfico de barras horizontal.
- **Desempeño por Sucursal:** Si hay múltiples sucursales, gráficos comparativos de ventas por sucursal.
- **Reportes Detallados:** Además de la vista resumida, permitir generar reportes específicos, por ejemplo:
- **Reporte de Ventas:** Lista de ventas/facturas filtrable por fecha, sucursal, cliente, con totales. Exportable a CSV/Excel.
- **Reporte de Inventario:** Listado de stock actual por sucursal, con valoración. Y posibilidad de ver movimientos de inventario en un rango de fechas (entradas/salidas).
- **Reporte de Servicios:** Listado de órdenes de servicio realizadas en un rango, con datos de cliente, vehículo, costo, técnico asignado, etc.
- **Reporte de Técnicos:** Métricas por técnico (órdenes completadas, horas registradas, valor generado en repuestos usados en sus órdenes, nivel de aprobación de sus presupuestos).
- **Reporte de Clientes:** Quizás quienes son los mejores clientes (más ingresos generados), cuántos vehículos tienen, etc., para apoyar campañas de marketing.
- **Gráficos Interactivos:** Usar librerías como **Chart.js** o **ECharts** para incorporar gráficos interactivos en la web. Esto permitirá, por ejemplo, hacer hover y ver datos exactos, filtrar por categorías, etc.
- **Actualización en Tiempo Real:** Si es posible, ciertos indicadores (como ventas del día) podrían actualizarse sin recargar la página, usando tecnologías como WebSockets o consultas periódicas en segundo plano, para que el administrador tenga info fresca. Por ejemplo, cuando se emite una nueva factura, que inmediatamente se refleje en el panel de ventas.
- **Configurabilidad:** Permitir al usuario seleccionar rangos de fechas personalizados en los reportes. Por ejemplo, ver las ganancias de un trimestre específico, o comparar año a año.
- **Exportación y Printing:** Además de visualizar, darle opción de descargar los gráficos o datos. Por ejemplo, botón "Exportar PDF" que genere un informe PDF con los gráficos (para una reunión, etc.), o al menos la data en CSV.
- **Seguridad:** Respetar los roles. Un gerente general podrá ver todos los datos de todas sucursales. Un gerente de sucursal quizás solo vea los de la suya en detalle. Un técnico normal probablemente no tenga acceso a los reportes financieros, pero tal vez sí a su propio desempeño básico. Ajustar visibilidad según rol.
- **Uso Interno:** Aclarar que este módulo es principalmente **interno** (no para clientes). Solo personal autorizado entra al dashboard de KPIs del negocio. Esto estará en la interfaz administrativa, no en el portal del cliente.
- **Alertas Derivadas de Datos:** Potencialmente, a futuro se puede agregar lógica para alertas, como "días sin ventas superiores a X" o "stock de producto Y no se ha movido en 6 meses" o "un técnico tiene más de N órdenes retrasadas". Esto excede lo básico, pero se deja la idea para mejoras.

Módulo de Inventario (Auditorías y Trazabilidad)

(Este punto amplía la funcionalidad del inventario con énfasis en auditorías, ya que el inventario básico ya fue descrito. Se puntualizan aquí mejoras o extras necesarios para control administrativo.)

El **módulo de auditoría de inventario** garantiza la **trazabilidad** completa de cada parte y facilita revisiones periódicas del stock. Es esencial para saber exactamente a quién se vendió cada pieza y cuándo, así como para detectar discrepancias entre el stock teórico y el físico. Características adicionales incluidas:

- **Registro de Transacciones Detallado:** Cada vez que una pieza sale o entra del inventario, generar un registro en una tabla de **movimientos de inventario** con toda la información: fecha/hora, tipo de movimiento (entrada por compra, salida por venta, salida para servicio, ajuste por auditoría, transferencia, etc.), cantidad, referencia (ej. número de factura de venta, número de orden de servicio, número de compra, etc.), usuario que ejecutó la acción, sucursal involucrada. Este historial es fundamental para auditoría.
- **Consulta de Historial por Ítem:** Desde la vista de un producto específico en inventario, permitir ver su historial de movimientos. Ejemplo: "Producto X - Filtro de aceite" -> historial: el 01/Nov entraron 20 unidades por compra #ABC, el 10/Nov salieron 5 unidades vendidas en factura #1001, el 15/Nov salieron 2 unidades para orden de servicio #200, etc. Así se puede seguir el rastro de cada lote.
- **Auditorías Físicas:** Incluir una función para **auditorías periódicas**. Esto puede funcionar de la siguiente manera:
 - Seleccionar una sucursal y realizar un "conteo físico" de ciertos o todos los productos.
 - Ingresar en el sistema los conteos físicos realizados.
 - El sistema compara el conteo físico contra el stock según sistema y muestra diferencias.
 - Permitir generar ajustes automáticos o manuales para cuadrar el inventario, con la debida justificación de auditoría. Por ejemplo, si en sistema decía 50 y físicamente hay 48, registrar salida de 2 por "merma/descuadre en auditoría fecha tal".
 - Generar un informe de auditoría con las discrepancias encontradas.
- **Control de Lote/Serie (Opcional):** Si algunas partes tienen número de serie (ej. baterías, partes costosas) o lote, y se requiere llevar control específico de a quién se le vendió cada número de serie, el sistema debe soportar asignar un número de serie a la salida. Esto es útil si luego hay reclamos o devoluciones; se sabe exactamente si la pieza devuelta corresponde a la vendida originalmente. Si esto se implementa, sería un sub-módulo de seriales/lotes.
- **Devoluciones de Producto:** Procesar devoluciones de repuestos (por ventas) o retornos de piezas no usadas en servicio:
 - Registrar la entrada de vuelta al inventario, referenciando la transacción original si posible.
 - Ajustar históricos de ventas si contablemente se requiere nota de crédito (ligado con facturación).
 - Este proceso debe también quedar en auditoría para que no luzca como "apareció stock de la nada".
- **Consultas de Responsabilidad:** Permitir consultas tipo "¿Quién vendió X pieza?" o "¿Qué piezas ha vendido el usuario Y?". Esto es útil ante irregularidades o para saber desempeño de vendedores. Se puede lograr porque los movimientos registran el usuario; entonces se hace un filtro por usuario o por producto.
- **Seguridad en Auditorías:** Probablemente solo gerentes o personal autorizado puede ejecutar auditorías y ajustes manuales de inventario. Asegurar que el sistema loguee estas acciones y quizás requiera doble autenticación o aprobación de un segundo usuario para ajustar grandes cantidades, para evitar mal uso.
- **Integridad de Datos:** Aplicar restricciones a nivel de base de datos para mantener integridad. Por ejemplo, movimientos de inventario no deberían permitir cantidades negativas resultantes; si se intenta sacar más de lo que hay, el sistema debe bloquearlo a menos que se trate de un ajuste deliberado (y en ese caso podría forzarse como ajuste). También, si se borra un producto (lo cual quizás esté prohibido si tiene movimientos), se debe manejar correctamente (mejor darlo de baja inactivo pero dejar sus registros históricos).

- **Herramientas de Auditoría:** Posibilidad de exportar todo el historial o ciertos movimientos a Excel para análisis externo por auditores. Y reportes como “Productos sin movimiento en X meses” para detectar inventario obsoleto.

En resumen, este módulo extra refuerza el inventario con capacidades de trazabilidad completa y control, asegurando que cualquier pieza pueda rastrearse desde que entró al almacén hasta que se vendió o usó, identificando quién estuvo involucrado en cada paso.

Módulo de Base de Datos y Administración de Datos

El sistema contará con un módulo (o al menos un conjunto de utilidades) para la administración de la **Base de Datos** y datos maestros del sistema. Dado que PostgreSQL es el corazón del almacenamiento, es importante planificar cómo se gestionará y mantendrá. Aspectos de este módulo:

- **Modelo de Datos (Diseño de la BD):** Definir claramente las tablas y relaciones principales:
 - *Usuarios:* tablas de usuarios (clientes, empleados) con roles.
 - *Vehículos:* tabla de vehículos con VIN, modelo, año, cliente propietario, etc.
 - *Repuestos:* tabla de productos de inventario con sus atributos.
 - *Inventario/Stock:* tabla para existencias por sucursal (puede ser una tabla separada que vincula producto + sucursal + cantidad).
 - *MovimientosInventario:* para auditoría de movimientos como se describió.
 - *Servicios/ManoObra:* tabla de servicios ofrecidos (catalogados).
 - *ÓrdenesServicio:* tabla principal de órdenes de taller (con estado, cliente, vehículo, fechas, técnico asignado...).
 - *DetalleOrdenServicio:* posiblemente tablas hijas para las líneas de cada orden: repuestos utilizados, mano de obra realizada.
 - *Facturas:* tabla de facturas emitidas (ventas), con su detalle en otra tabla de líneas de factura (producto/servicio, cantidad, precio).
 - *Compras:* tabla de compras a proveedores, con detalle.
 - *Pagos:* tablas para pagos recibidos (caja) y pagos hechos (a proveedores), si se maneja.
 - *Sucursales:* tabla de sucursales/tiendas.
 - *Proveedores, Clientes:* tablas para terceros.
 - *Notificaciones/Marketing:* tablas para mensajes a enviar o historial de notificaciones.
 - *Media/Videos:* tabla para referencias a videos subidos en el módulo de señalización (almacenando ruta o nombre de archivo, metadatos).
 - Etc. (Se puede detallar cada tabla en un anexo, pero en general estas son las entidades).
- **Configuración General:** Una tabla o mecanismo para almacenar configuraciones de la aplicación, por ejemplo:
 - Datos de la empresa (nombre, RTN, logo) para imprimir en facturas.
 - Parámetros fiscales (ISV%, CAI actual, fechas).
 - Configuraciones de notificaciones (templates de correo, etc.).
 - Esto permite cambiar ciertos comportamientos sin tocar código.
- **Backup y Restauración:** Proveer herramientas para respaldo de la base de datos. Podría ser simplemente documentación para usar `pg_dump`, pero idealmente integrar en el sistema:
 - Un botón o script para generar backup .sql periódicamente (por ejemplo, un cron job diario que guarde copia).
 - Instrucciones claras en el manual de cómo restaurar en caso de desastre.

- **Migraciones de Esquema:** Como Codex generará el código, se deben incluir migraciones/esquemas (por ejemplo usando Sequelize CLI, Alembic, Django migrations, etc.) para que la base de datos se pueda crear y actualizar fácilmente. El instalador se apoyará en esto para construir las tablas.
- **Carga de Datos Iniciales:** Posiblemente el instalador o módulo DB cargue datos base:
 - Usuarios iniciales (admin),
 - Sucursal inicial,
 - Algunos datos de ejemplo (repuestos genéricos, etc.),
 - Catálogo de servicios (mano de obra) predefinidos. Esto para que el sistema arranque operable.
- **Interfaz de Administración (Opcional):** Si se desea, crear una pequeña interfaz dentro de la aplicación para tareas de DB:
 - Por ejemplo, un módulo de **Administración del Sistema** donde solo super-admin entra, con opciones: "Respaldo de BD", "Restaurar BD" (esta última con cuidado), "Ver registros del sistema" (logs).
 - También para gestionar datos maestros: p.ej. actualizar catálogo de servicios, listar todas las sucursales y editarlas, etc., que no caen en otros módulos.
- **Logs del Sistema:** Configurar que el backend registre logs de eventos importantes o errores en archivos (o tabla de logs). Esto ayuda en diagnósticos. Un apartado del módulo de base de datos/ administración podría permitir ver los logs recientes (por ejemplo, errores de integridad, intentos de login fallidos, etc.).
- **Conexión y Pooling:** A nivel técnico, el backend usará un pool de conexiones a PostgreSQL para eficiencia. Configurar parámetros adecuados (número de conexiones, timeouts) considerando la cantidad de usuarios esperados.
- **Mantenimiento:** Detallar en manual cómo actualizar el esquema si en un futuro se agregan más campos o tablas. Es importante que Codex genere el proyecto con herramientas de migración para facilitar esto.
- **Seguridad de la DB:** Asegurar que las credenciales de la base de datos estén protegidas (por ejemplo, en archivos de configuración no subidos a GitHub, o usando variables de entorno). Incluir en la documentación la necesidad de cambiar contraseñas por defecto, etc. Además, garantizar que la comunicación con la base de datos esté restringida (si el servidor web y DB están en la misma máquina, socket local; si no, restringir puertos).

Módulo de Marketing (Notificaciones y Promociones)

El módulo de **Marketing** es sencillo pero útil para mantener el contacto con los clientes desde la aplicación misma. Su fin es enviar notificaciones sobre promociones, recordatorios de mantenimiento preventivo y novedades del taller. Características principales:

- **Gestión de Campañas Internas:** Permitirá crear pequeñas **campañas de notificación** hacia los clientes. Ejemplos:
 - Envío de un descuento especial ("Descuento del 10% en cambios de aceite este mes de aniversario").
 - Recordatorio de servicio ("Han pasado 6 meses desde el último cambio de aceite de tu vehículo, ¡agenda una cita!").
 - Anuncio de nuevos servicios o productos ("Ahora vendemos llantas de marca X, pregúntanos por precios especiales").
- **Segmentación Básica:** Posibilidad de elegir a quién enviar:
 - A todos los clientes.

- A clientes específicos (por ejemplo, por modelo de vehículo: "promoción para SUVs", o por fecha de último servicio, etc.).
- Esta segmentación básica se puede lograr mediante filtros en la lista de clientes: por fecha de último servicio, tipo de servicio recibido, etc. En primera versión, incluso enviar a todos es suficiente, pero dejar la estructura para filtrar más adelante.
- **Canales de Notificación:** Decidir cómo se entregarán los mensajes:
- **Dentro de la App (In-App):** Al estilo de un pequeño anuncio que aparece en el portal del cliente cuando inicia sesión (o una sección de mensajes/promociones).
- **Correo Electrónico:** Si se tiene el email del cliente, se podría integrar con un servidor SMTP o servicio de email (como SendGrid, etc.) para enviar correos masivos con la promoción. Inicialmente se puede implementar usando una cuenta de correo del taller.
- **SMS/WhatsApp:** Sería ideal para recordatorios de mantenimiento, pero requeriría integración con APIs externas (Twilio, etc.). Podría ser un futuro upgrade; de momento se documenta como posibilidad.
- **Plantillas de Mensaje:** Permitir diseñar un mensaje con variables, por ejemplo: "Hola [NombreCliente], tu vehículo [ModeloVehiculo] está próximo a [KilometrajeProximoServicio] km. Te recomendamos un chequeo. Promo: ...". Estas plantillas simples pueden almacenarse y luego el sistema las rellena para cada destinatario. O se puede enviar un mensaje genérico sin personalización.
- **Programación de Envíos:** Para recordatorios periódicos (ej. cambios de aceite cada 3 meses), sería útil que el sistema automáticamente generara un recordatorio. Por ejemplo, al cerrar una orden de cambio de aceite, programar en el sistema que 3-6 meses después se envíe un recordatorio al cliente. Esto implica un pequeño **job scheduler**. Se puede implementar un script que corra diariamente buscando eventos programados para hoy (cronjob) y dispare los mensajes correspondientes.
- **Historial de Marketing:** Llevar registro de qué notificaciones se enviaron, cuándo y a quién. Así se puede saber cuántos clientes fueron notificados de X promoción y eventualmente medir efectividad (si regresaron al taller).
- **Opt-in/Opt-out:** Por respeto al cliente, permitir que un cliente pueda optar por no recibir mensajes promocionales (esto puede ser un campo en su perfil). El sistema debe respetar eso y excluirlos de campañas masivas que no sean estrictamente operativas (como sus recordatorios personales, que igual podría opt-out).
- **Uso Interno:** Este módulo es operado por personal de administración o marketing del taller. Probablemente vía la misma interfaz administrativa (no de cara al cliente). Podría ser una sección donde se redacta el mensaje, se elige destinatarios y se envía.
- **Integración con Portal Cliente:** Las promociones relevantes pueden reflejarse en el portal (como se mencionó, una sección de notificaciones para el cliente). El marketing module gestionaría qué mensajes aparecen allí y por cuánto tiempo.
- **Pequeño Alcance Inicial:** Dado que es "pequeño módulo de marketing", se puede iniciar con algo básico:
 - Un formulario para escribir un mensaje y enviarlo a todos los clientes (via email y guardarlo para mostrar en portal).
 - Sin mucha automatización inicialmente. Más adelante se le añade programación y segmentación.
 - Lo importante es sentar las bases para comunicación al cliente desde el sistema.

Módulo de Señalización Digital (OpenSignage)

El módulo de **OpenSignage** permitirá gestionar contenido multimedia (particularmente videos) para proyectarlos en pantallas dentro del taller u otros locales, a modo de cartelería digital. La idea es que el taller pueda subir sus anuncios o videos informativos al sistema y este genere enlaces o una forma fácil de reproducirlos en televisores o monitores. Elementos clave de este módulo:

- **Subida de Videos:** Una interfaz para que el usuario admin pueda **subir archivos de video** (formatos comunes como MP4) al servidor. Posiblemente limitar el tamaño o recomendar cierta duración/resolución para que sea manejable. Los videos subidos se almacenarán en el sistema de archivos del servidor o en la base de datos (lo más eficiente es en el sistema de archivos, guardando solo la ruta en la BD).
- **Catálogo de Medios:** Listar los videos subidos con detalles: nombre, fecha de subida, duración (si se puede obtener), y tal vez una descripción o etiqueta (ej. "Promoción Cambio de Llantas", "Video Institucional", etc.).
- **Generación de Link/Reproducción:** Por cada video o lista de reproducción, generar un **enlace público o interno** que muestre el video a pantalla completa:
- Podría ser una ruta como `/signage/player?id=XYZ` que abre una página web minimalista con el video en bucle o la lista en secuencia. Esa página se puede poner en un navegador en la TV (muchos televisores smart permiten abrir URLs) o en una Raspberry Pi conectada a un monitor.
- Si se desea más control, se podría generar un código aleatorio o clave para que no cualquiera encuentre el video, pero como es para pantallas públicas tampoco es muy sensible.
- **Listas de Reproducción:** Para mayor flexibilidad, permitir crear **playlists** de varios videos (ej. rotar 3 anuncios). El link generado entonces reproduce todos en ciclo continuo. La interfaz de administración dejaría ordenar los videos en la lista y elegir si hay tiempo de pausa entre ellos, etc.
- **Configuración de Pantallas:** (Opcional) Si se quiere gestionar múltiples pantallas con contenidos distintos, se puede crear la noción de **Pantalla** o **Display**:
- Cada pantalla tiene un código o nombre (ej. "Sala de espera", "Recepción", "Exterior").
- Se puede asignar una playlist distinta a cada pantalla.
- El enlace generado podría ser específico por pantalla (ej. `/signage/player/recepcion` muestra la playlist asignada a "Recepción").
- Esto ayuda si se tiene varias sucursales, cada una con su pantalla mostrando contenido local quizás.
- **Interfaz del Reproductor:** La página que reproduce el video debe ser simple, probablemente en el frontend usar HTML5 video element or a JS library for video, set to fullscreen. Configurar para repetir eternamente. Tal vez soporte modos: reproducir en bucle un solo video, o secuencia de varios.
- **Actualización en Vivo:** Cuando se agregue un nuevo video o cambie una playlist, sería bueno que las pantallas reflejen el cambio automáticamente tras terminar su ciclo, en lugar of necesitar reinicio manual. Esto podría hacerse mediante websocket o una simple recarga programada para buscar cambios. Este detalle es mejora adicional pero se menciona como posible.
- **Uso de Recursos:** Asegurarse de servir los videos eficientemente. Si son muy pesados, considerar streaming progresivo. Pero dado que es en LAN o controlado, simplemente almacenarlos y servirlos está bien. Si la cantidad de videos es grande, monitorear espacio en disco.
- **Seguridad:** Este módulo es principalmente interno. Si se prefiere, asegurar que los enlaces de reproducción tengan algún token para que no cualquiera escanee la IP del server y vea videos no destinados. Pero tampoco es crítico si el contenido no es secreto. La subida de videos sí debe ser restringida a roles autorizados.

- **Integración:** El módulo de marketing podría tie-in aquí: algunos videos pueden ser parte de campañas o promociones que también se notifican a clientes. Pero en general, OpenSignage funciona autónomo para displays físicos.
- **No Reinención de la Rueda:** Se llama "OpenSignage" quizá inspirándose en algún software existente. Si hubiera bibliotecas para gestión de digital signage, se podría integrar, pero probablemente con las propias herramientas web es suficiente. Documentar este módulo claramente para que Codex genere la funcionalidad de almacenamiento y reproducción de medios.

Módulo de Sucursales (Multi-Tienda y Puntos de Venta)

El módulo de **Sucursales** habilita la operación del sistema en múltiples ubicaciones físicas (talleres o tiendas) bajo la misma empresa. Cada sucursal puede tener su propio inventario separado, sus propias ventas y eventualmente ciertas configuraciones particulares, pero todas comparten una base de datos central (separando datos por un identificador de sucursal). Funcionalidades y consideraciones:

- **Gestión de Sucursales:** Interfaz para que un administrador pueda **crear, editar o eliminar** sucursales. Datos a guardar por sucursal: nombre, código, dirección, teléfonos, quizás datos fiscales si requieren emitir facturas con distinta razón social (pero si es misma empresa, solo cambia dirección).
- **Inventario por Sucursal:** Como mencionado, el inventario tendrá cantidades por sucursal. En la UI de inventario, se podrá filtrar por sucursal o ver consolidado. Al agregar un nuevo producto, se puede inicializar en una sucursal específica o en todas con 0. Transferir stock entre sucursales es importante:
 - Una función de **transferencia**: seleccionar producto, cantidad, sucursal origen y destino. Esto crea dos movimientos (salida en origen, entrada en destino) ligados.
 - Al buscar repuestos para una orden de servicio o venta, idealmente mostrar prioridad a la sucursal actual del usuario, pero también indicar si en otras sucursales hay stock (por si se puede traer).
- **Ventas/Facturación por Sucursal:** Cada factura debe quedar asignada a la sucursal que la emite. Esto afecta la numeración fiscal si cada sucursal tiene su propio rango autorizado. Implementar en el módulo de facturación la selección o pre-asignación de serie según sucursal.
- **Usuarios por Sucursal:** Los usuarios empleados deberían estar asociados a una o múltiples sucursales:
 - Un técnico normalmente pertenece a una sucursal (donde trabaja), igual un almacenista o vendedor.
 - Un administrador de sucursal quizás solo ve datos de su sucursal.
 - Usuarios corporativos (dueño, contable general) podrían ver todo.
 - Clientes podrían estar asociados a una sucursal preferida (p.ej. donde suelen asistir), pero en general podrían ir a cualquiera; su vehículo historial sería visible en cualquier sucursal al buscar por VIN.
- **Sucursales y Cajas/Puntos de Venta:** Si una sucursal tiene varias **cajas registradoras** o áreas de facturación, se puede considerar sub-division. Sin embargo, eso quizá complica mucho. Podría bastar con distinguir sucursales a nivel de facturación. Si realmente se requiere múltiples cajas por sucursal, se podría añadir un identificador de caja en la factura (p.ej. Factura #000-001-01... para caja 1 y #000-001-02... para caja 2, etc. según como SAR lo autorice). De momento, se puede obviar o dejar preparado.
- **Reportes por Sucursal:** El módulo de gráficos y reportes debe filtrar por sucursal:
 - Permitir obtener indicadores por cada sucursal individual, y comparativos.

- Esto implica que casi todas las consultas (ventas, inventario, servicios) incluyan un parámetro de sucursal.
- **Comunicación entre Sucursales:** Si un cliente va con su vehículo a otra sucursal, se debería poder consultar su historial igualmente. Por lo tanto, la data de clientes, vehículos y historiales es **global compartida** (no duplicar por sucursal). Un vehículo es atendido en múltiples sucursales pero es el mismo registro con referencias a dónde fue atendido en cada servicio.
- **Configuraciones Regionales:** Si las sucursales estuvieran en distintas ciudades, tal vez monedas o impuestos diferentes podrían aplicar, pero asumiendo mismo país, eso no varía. Solo tener en cuenta si alguna sucursal tiene un impuesto municipal adicional, etc., aunque no se mencionó. De ser el caso, poner campo para tasa municipal configurable por sucursal.
- **Desempeño:** Con varias sucursales operando concurrentemente, hay que asegurar que la aplicación sea eficiente en concurrencia, especialmente en inventario (evitar condiciones de carrera cuando dos sucursales vendan la última unidad a la vez, por ejemplo). Utilizar transacciones o locks a nivel DB para movimientos críticos de stock.
- **Soporte Multi-tenant vs Single DB:** Aquí optamos por single database multi-tenant (distinguido por IDs). Es más sencillo de manejar y ideal para consolidar reportes. Documentar bien en el código qué tablas tienen campo sucursal_id y cómo se filtra en cada consulta dependiendo del contexto del usuario logueado.
- **Instalación para Sucursales Nuevas:** Incluir en el manual cómo agregar una sucursal nueva al sistema (usando la interfaz, o manualmente si DB). Y si hubiera componentes instalados localmente (por ej. un PC en la sucursal con la app), aclarar si solo se necesita internet hacia el servidor principal. Probablemente la arquitectura es centralizada, así que con conexión todo reside en el server central. Si se necesitara offline mode para alguna sucursal, eso complicaría bastante, así que no se considera en esta fase.

Módulo de Usuarios y Seguridad (Gestión de Usuarios)

El módulo de **Usuarios** abarca la administración de todas las cuentas de usuario del sistema, sean empleados del taller en distintos roles o clientes finales, así como la seguridad relacionada (roles, permisos, autenticación). Funcionalidades:

- **Gestión de Usuarios Internos:** Crear, editar, eliminar cuentas de empleados:
- Campos: nombre, email/usuario, contraseña (inicial, luego se cambia), rol, sucursal asignada (si aplica), estado (activo/inactivo).
- Roles predefinidos: Administrador General, Gerente de Sucursal, Técnico, Recepcionista, Almacén/Inventario, Contador, Cajero/Facturador, Marketing, etc. (Se pueden ajustar las categorías, incluso combinarlas).
- Cada rol tendrá permisos específicos para módulos. Por ejemplo, Técnico: acceso a módulo técnico y su info, pero no a contabilidad; Inventario: acceso a gestión de inventario pero quizás no a modificar facturas, etc. Estos permisos pueden definirse en un archivo de configuración o en la base de datos. Para simplificar, se puede codificar en el backend las reglas de qué rol puede invocar qué endpoints, o usar un sistema de permisos más dinámico.
- **Gestión de Clientes (Usuarios Externos):** Desde esta interfaz (o quizá desde módulo "Garage"), permitir crear cuentas de cliente:
- A veces los clientes se registrarán solos vía portal. En tal caso, un administrador debería poder revisar esos registros y validarlos (opción de aprobar nuevos registros o dejarlos automáticos).

- Vincular cliente con sus vehículos: un admin puede asociar un vehículo existente a un cliente si está duplicado, etc.
- Reset de contraseña o asistencia a clientes que no puedan entrar.
- **Autenticación:** Implementar login seguro. Si se usa JWT para la SPA, el módulo de usuarios manejará la generación de tokens. Alternativamente, cookies de sesión si es multi-page. Dado que es un sistema privado, JWT con expiración y refresh tokens es una buena opción para API.
- Incluir funciones de **recuperación de contraseña** (e.g., el clásico "¿Olvidó su contraseña?" enviando email con enlace de reset). Esto requerirá configuración de email SMTP.
- **Roles y Permisos Detallados:** Posiblemente una pantalla donde un Admin puede cambiar permisos finos, aunque no es imprescindible al inicio. Podría definirse que los roles están fijos en código y listo. Documentar los roles claramente:
- Administrador General: acceso total a todos módulos y configuraciones.
- Gerente Sucursal: acceso a todo de su sucursal (inventario, ventas, órdenes) pero no a otras sucursales ni a config global.
- Técnico: acceso a Órdenes de Servicio, puede ver historiales de vehículos, y tal vez sus propias métricas, pero no finanzas ni facturas.
- Inventario: acceso a Inventario, puede ver productos y movimientos, hacer entradas/salidas, pero no tocar facturas o contabilidad.
- Cajero/Facturador: acceso a Facturación (crear facturas), ver listados de ventas de su sucursal, y a lo sumo consultar inventario para facturar; no modifica inventario ni contabilidad.
- Contador: acceso a módulo Contabilidad (compras, libros), puede ver facturas de todas sucursales, etc., pero tal vez no modificar órdenes de servicio.
- Marketing: acceso al módulo Marketing y tal vez a gráficos de clientes, pero no a operaciones del taller.
- Cliente: acceso solo a su portal, sus datos.
- **Auditoría de Usuarios:** Registrar log de accesos y acciones sensibles. Por ejemplo, quién eliminó o creó un usuario, quién cambió un rol, etc. Este log de seguridad puede guardarse en una tabla y mostrarse al super-admin en caso de incidentes.
- **Sesiones y Control:** Si se usa JWT, tener mecanismos de invalidar tokens cuando un usuario es deshabilitado (lista negra de tokens). Si se usa sesiones, manejar expiración e idealmente opción de cerrar sesión remota.
- **Protección de Datos:** Asegurarse que datos de usuario (como contraseñas) están hasheadas, y que la comunicación se hace por HTTPS (eso es configuracional al desplegar en servidor web Nginx/Apache).
- **Escalabilidad de Usuarios:** Soportar un número creciente de usuarios sin degradación. Uso de índices en BD (por email, etc.) para autenticar rápido.
- **Experiencia de Usuario (Admin):** Hacer fácil el uso de esta sección: un admin debería rápidamente poder, por ejemplo, resetear la contraseña a un técnico que la olvidó, o cambiar al técnico Juan de sucursal A a B si se transfiere. Esos cambios deben reflejarse en permisos de datos inmediatamente.
- **Integración:** Casi todos los módulos dependerán de saber quién es el usuario autenticado y su rol, por lo que este módulo es central. El backend debe incluir validación de permisos en cada endpoint (por ejemplo, usando middleware en Node/Express para JWT + role checking). En el frontend, también ocultar o mostrar secciones del menú según el rol del usuario logueado, para mejor usabilidad.

Integración entre Módulos y Flujo General del Sistema

Después de detallar cada módulo, es importante describir **cómo interactúan** entre sí en el flujo de trabajo cotidiano. A continuación, se presentan algunos escenarios y cómo los módulos colaboran en cada caso:

- **Ingreso de Vehículo y Reparación (Flujo Completo):**
 - **Recepción:** El recepcionista o técnico crea una Orden de Servicio en el **Módulo Técnico**, registrando datos del cliente/vehículo (integrado con Usuarios/Vehículos) y el problema reportado.
 - **Diagnóstico:** El técnico realiza diagnóstico y agrega repuestos y mano de obra estimada desde el Módulo Técnico. Para ello consulta el **Módulo de Inventario** (verifica disponibilidad de piezas) y el catálogo de **Mano de Obra**. Genera un presupuesto.
 - **Aprobación Cliente:** El presupuesto viaja al **Portal Garage** (cliente), notificación manejada por **Marketing/Notificaciones**. El cliente lo aprueba en el portal.
 - **Reparación:** Con la aprobación, el técnico solicita piezas al almacén. El **Módulo de Inventario** recibe la solicitud, el encargado la atiende; se registran salidas de stock con referencia a la orden. El técnico realiza el trabajo registrando progreso en el sistema (Módulo Técnico actualiza estado).
 - **Finalización:** Técnico marca la orden como lista. Se genera la factura en **Módulo de Facturación** con los ítems (piezas y labor) de esa orden. Inventario ya está descontado, pero si algo faltaba se descuenta aquí final. Se imprime la factura, se entrega al cliente con el vehículo.
 - **Pago y Registro:** Si el pago es de contado, se registra la entrada en contabilidad (Caja/Bancos) por Facturación->Contabilidad. Si es crédito, Contabilidad registra cuenta por cobrar. La orden de servicio se cierra completamente.
 - **Historial:** El **Historial del vehículo** (en Portal Garage) se actualiza con esa reparación realizada. Cliente puede verlo cuando quiera.
 - **Datos para Gráficos:** Toda esta operación alimenta Gráficos: venta sumada, repuestos más usados, rendimiento del técnico, etc., que el gerente puede revisar después.
- **Venta Directa de Repuesto (Mostrador o Portal):**
 - Un cliente solicita comprar una pieza sin servicio de taller. Puede ser un cliente que llega a mostrador o que la encarga vía **Portal Garage**.
 - El vendedor busca el repuesto en el **Módulo de Inventario** (filtra por código o por categoría, o en portal el cliente la ubica vía 3D).
 - Se añade al carrito (Portal) o a la factura directamente (en tienda). El sistema verifica stock en la sucursal actual; si no hay, informa si otra sucursal tiene (posible acción: pedir transferencia, o solicitar pedido especial).
 - **Facturación:** Se genera factura de venta del repuesto en **Módulo Facturación** con impuestos. Cliente paga, se cierra la venta.
 - **Inventario:** Stock reducido automáticamente. Movimiento registrado (salida por venta # factura).
 - **Contabilidad:** Registra la venta e ingreso.
 - **Entrega:** Si fue en tienda, se entrega la pieza físicamente. Si fue online, se coordina envío o pickup (puede requerir notificación al cliente de que su pedido está listo, eso entra en flujo de marketing/notificaciones).
- **Compra de Stock a Proveedor:**
 - El administrador de inventario detecta con los **Reportes** que cierto repuesto está bajo mínimo. Crea una **Orden de Compra** en el **Módulo Contabilidad/Compras**.
 - Al recibir la mercancía, ingresa la factura del proveedor en el sistema: eso aumenta el **Inventario** (entrada de stock) y registra una cuenta por pagar en **Contabilidad**.

- Inventario guarda el movimiento de entrada (con referencia a compra). Si algún pedido de cliente estaba pendiente por falta de stock, ahora se puede completar (podríamos generar notificación "pieza X disponible" vía Marketing si quisiéramos).
- Contabilidad refleja el nuevo valor de inventario y pendiente de pago al proveedor. Cuando se paga, se registra salida de dinero.
- **Auditoría de Inventario:**
 - Gerente inicia un proceso de auditoría en **Módulo de Inventario (Auditorías)**. Selecciona sucursal, imprime o toma nota de cantidades actuales registradas.
 - El personal cuenta físicamente y detecta diferencias. Ingresan los recuentos en el sistema.
 - El sistema calcula ajustes; el gerente los aprueba, generando movimientos de ajuste (ej. -2 piezas X por pérdida).
 - Se investigan las diferencias usando el historial: por ejemplo, se ve que cierto usuario registró una salida manual dudosa. Podría haber implicaciones de seguridad (por eso log de usuarios es útil).
- **Campaña de Marketing (Recordatorio):**
 - El sistema (vía un job programado en el **Módulo Marketing**) detecta que han pasado 6 meses desde que 5 clientes hicieron cambio de aceite.
 - Automáticamente envía emails a esos 5 (y crea notificación en Portal) diciendo "Te toca cambio de aceite, ven al taller con 10% de descuento".
 - Algunos clientes responden agendando cita o por lo menos vienen. Esto incrementa órdenes de servicio, que siguen el flujo normal de técnico->factura.
 - El administrador puede ver en reportes cuántos clientes vinieron gracias a la campaña (tal vez manualmente verificando fechas).
- **Uso de OpenSignage:**
 - El usuario de marketing sube un nuevo video promocional "Servicio de alineación descuento". Crea una playlist "Pantalla Recepción" con ese video y otro más institucional.
 - En la recepción del taller, hay un Smart TV con un navegador abierto apuntando al URL del reproductor de "Pantalla Recepción".
 - El contenido se actualiza y comienza a mostrar el nuevo video en bucle. Clientes en la sala de espera lo ven.
 - Esto complementa la estrategia, aunque no interactúa con los otros módulos de datos, es independiente pero administrado dentro del mismo sistema para conveniencia.

Estos escenarios muestran que el sistema funciona de forma integrada: los módulos no son aislados, sino que comparten datos y se activan unos a otros en secuencia. Este blueprint está pensado para que Codex genere código con módulos bien acoplados a través de API internas o llamadas de funciones, manteniendo consistencia en IDs (por ejemplo, la orden de servicio #200 referenciada en factura #1005, etc.).

Además, la interacción multi-módulo implica que se deben escribir pruebas de integración para asegurarse de que el flujo completo funcione (por ejemplo, un test que cree una orden, apruebe, facture, y verifique que inventario y contabilidad se actualizaron correctamente).

Despliegue, Instalación y Pruebas

Para que el sistema esté disponible y funcionando correctamente, se deben seguir ciertas pautas de despliegue en entorno Ubuntu, así como la ejecución de pruebas para validar funcionalidades. Este apartado funciona como manual de instalación y aseguramiento de calidad:

- **Instalación en Ubuntu (Pasos Automáticos y Manuales):**
- **Prerequisitos:** Ubuntu Server 20.04+ actualizado, con acceso a internet. Instalar paquetes esenciales: `git`, `node.js` (si Node es el backend elegido, se sugiere Node 18 LTS), `npm`, `python3` (si se usan scripts Python o un backend en Python), y `postgresql`. También `nginx` si se servirá la aplicación a través de un servidor web reverse proxy.
- **Descarga del Código:** Descomprimir el archivo ZIP proporcionado o clonar el repositorio de GitHub en el servidor. Se ubicará, por ejemplo, en `/opt/taller-app`.
- **Configuración de Base de Datos:** Instalar PostgreSQL (`sudo apt install postgresql`). Crear un usuario de base de datos (ej. `talleruser`) y una base de datos (ej. `tallerdb`) con su password. Alternativamente, el instalador automático preguntará por credenciales de superuser para crear esto vía script.
- **Variables de Entorno:** Configurar variables necesarias (pueden estar en un `.env` file):
 - `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS` para la conexión a la base de datos.
 - `NODE_ENV` en producción, `PORT` para el servidor web, etc.
 - Claves para JWT (`JWT_SECRET`), parámetros de email SMTP if needed, etc.
 - Configuraciones de factura (prefijo, CAI, etc., aunque esos también podrían ponerse vía interfaz luego).
- **Instalación de Dependencias:** Ejecutar el gestor de paquetes correspondiente:
 - Para Node backend: `npm install` en la carpeta del backend (instalará Express, ORM, JWT lib, etc.).
 - Para frontend: `npm install` o `yarn` en carpeta frontend (instalar React/Angular libs, Babylon.js, Chart.js, etc.).
- **Compilación/Build (Frontend):** Si se trata de React/Angular, ejecutar `npm run build` para generar la versión estática optimizada del frontend. Esto creará archivos estáticos que el backend puede servir o que Nginx puede servir directamente.
- **Migrar Base de Datos:** Ejecutar las migraciones para crear las tablas. Dependiendo del stack, esto podría ser:
 - Node+Sequelize: `npx sequelize db:migrate` (o un script `npm run migrate`).
 - Python+Django: `python manage.py migrate`.
 - O importar un SQL dump si se incluye. Lo ideal es migración programática para distintas instalaciones.
- **Datos Iniciales:** Cargar datos iniciales:
 - Crear un usuario admin por defecto. Quizá el instalador pida un email y password para el superusuario inicial.
 - Cargar catálogos básicos (servicios, quizá algunos repuestos de muestra, roles). Esto se puede hacer con un script `npm run seed` por ejemplo.
- **Configuración de Servicios:** Configurar el backend para que corra como un servicio del sistema (por ej. crear un servicio systemd `taller.service` que ejecute `npm start` en la carpeta backend, para que inicie al arrancar el servidor y se mantenga).

- **Configuración Nginx (opcional):** Usar Nginx como reverse proxy: configurar un sitio que escuche en el puerto 80/443, pase tráfico a `http://localhost:3000` (si backend Node en 3000). También servir archivos estáticos/medios (por ejemplo los videos de OpenSignage) desde una carpeta segura.
- **Firewall:** Asegurar que los puertos abiertos son los necesarios (80/443). Cerrar accesos directos a DB desde fuera (que Postgres escuche solo local).
- **Docker Abstención:** No usar contenedores; todo instalado en el host significa los paquetes se instalan globalmente. Documentar bien dónde están las cosas para mantenibilidad (por ej. código en /opt, logs en /var/log/tallerapp.log, etc.).
- **Pruebas (Testing):**
- **Pruebas Unitarias:** El proyecto incluirá pruebas unitarias para funciones críticas. Por ejemplo:
 - Funciones de cálculo de impuestos, formatos de factura.
 - Lógica de aprobación de presupuesto (que marque estados correctamente).
 - Cálculos de reportes (dado ciertos datos de entrada, produce salidas esperadas).
- **Pruebas de Integración:** Usando herramientas como Jest + Supertest (en Node) o PyTest (Python) o similar, se simularán llamadas a la API:
 - Test de flujo completo: crear orden -> aprobar -> finalizar -> verificar que existan registros de factura, movimientos de inventario, etc., coherentes.
 - Test multi-sucursal: crear algo en sucursal A y B, asegurar separación.
 - Test de permisos: un técnico tratando de acceder a endpoint de contabilidad debe recibir prohibido.
- **Pruebas de Interfaz:** Opcionalmente, se podrían incluir pruebas end-to-end con Selenium o Playwright para asegurarse de que las pantallas funcionan (eso podría ser mucho, quizás se omite, pero se deja posibilidad).
- **Revisión Manual:** Indicar en el manual técnico algunos casos de prueba manual para el implementador:
 - Crear un cliente y vehículo, generar una orden, probar todo flujo.
 - Crear una factura de venta directa.
 - Probar hacer una auditoría de inventario.
 - Probar envío de una notificación de marketing de prueba a uno mismo.
- **Entorno de Pruebas:** Se puede tener una base de datos separada para testing, o usar transacciones revertibles en los tests para no contaminar datos reales.
- **Documentación y Soporte:** En el README o manual incluir:
 - Resumen de funcionalidades (mucho viene de este blueprint).
 - Cómo usar cada módulo brevemente para el usuario final (puede ser un manual de usuario aparte).
 - Cómo reportar issues o cómo actualizar el sistema (pull de GitHub y re-run migrations, etc.).
 - Asegurarse que el blueprint/código quede bien documentado en comentarios, para facilitar mantenimiento.
- **Entrega Final:** El resultado que Codex genere deberá ser subido a un repositorio GitHub. Se verificará que:
 - El código está estructurado claramente en carpetas (backend, frontend, docs, etc.).
 - Existan los archivos de configuración (.env.example, etc.).
 - El instalador (script .sh o instrucción) funciona en una máquina limpia Ubuntu (esto se probará).
 - El sistema corre sin errores y pasa las pruebas automatizadas.
 - Cualquier dependencia externa (e.g. email SMTP account) esté abstracta para que el implementador sustituya con sus credenciales.
 - No se almacenan secretos en el repo (buenas prácticas).

Una vez instalado, el sistema debería estar listo para usar, y el siguiente paso sería realizar pruebas con usuarios reales en un entorno controlado, ajustar detalles de usabilidad, y luego ponerlo en producción en el taller.

Conclusión

Este **blueprint y manual técnico** cubre todos los componentes solicitados para el sistema integral de gestión de un taller automotriz, incorporando módulos de inventario, mano de obra, portal del cliente con modelo 3D (Garage), facturación conforme al SAR de Honduras, contabilidad básica, módulo para técnicos, visualización gráfica de indicadores, auditoría de inventario, administración de base de datos, marketing interno, señalización digital, multi-sucursal y gestión de usuarios.

La arquitectura propuesta con PostgreSQL y una stack web moderna garantiza robustez y escalabilidad, evitando complicaciones de contenedores al instalar directamente en Ubuntu. Se han pensado detalles de integración entre módulos para que Codex (u otro equipo de desarrollo) pueda implementar una solución cohesionada y funcional al 100%.

Con esta guía, el desarrollador AI (Codex) puede proceder a codificar el sistema paso a paso, realizando pruebas en cada etapa, tomando como referencia proyectos similares (como cualquier avance previo del proyecto "Smart504" mencionado, si está disponible) pero adaptando y mejorando según lo aquí descrito. Al finalizar, el taller contará con una herramienta poderosa que digitaliza sus operaciones de punta a punta, mejora la eficiencia interna y la experiencia de sus clientes, todo respaldado por un diseño técnico sólido. ¡Manos a la obra!
